



Hibernate ORM trong Java

Giải pháp quản lý dữ liệu hiệu quả cho ứng dụng Java Enterprise

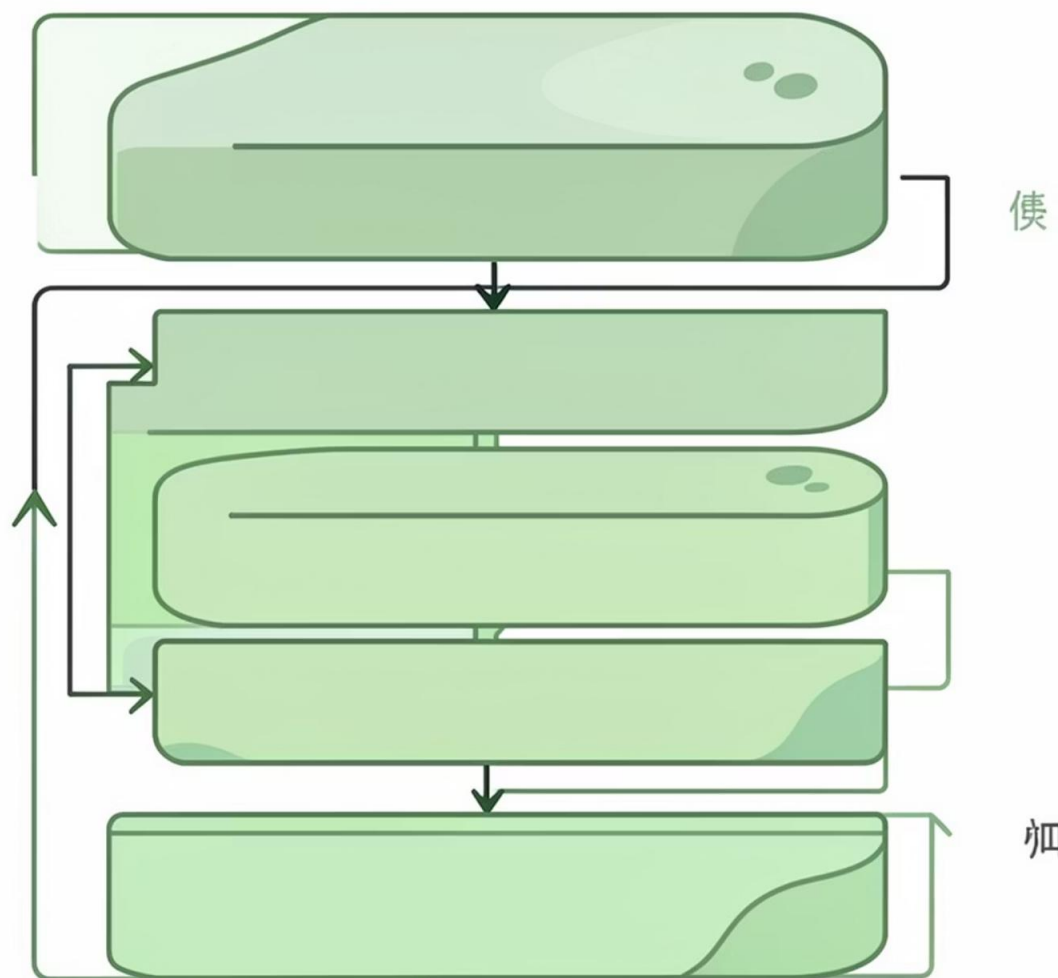
Tại sao cần ORM?

Vấn đề với JDBC

- Viết nhiều code lặp lại cho mỗi truy vấn SQL
- Phải tự chuyển đổi giữa kiểu dữ liệu Java và SQL
- Quản lý kết nối và tài nguyên phức tạp
- Code phụ thuộc vào CSDL cụ thể
- Khó bảo trì và mở rộng

Lợi ích của ORM

- Tự động chuyển đổi dữ liệu object ↔ table
- Giảm đáng kể lượng code cần viết
- Quản lý tài nguyên tự động và hiệu quả
- Độc lập với loại CSDL sử dụng
- Code rõ ràng và dễ bảo trì hơn



Hibernate là gì?



Framework ORM

Giải pháp mã nguồn mở cho ánh xạ dữ liệu Java sang cơ sở dữ liệu quan hệ



Tầng trung gian

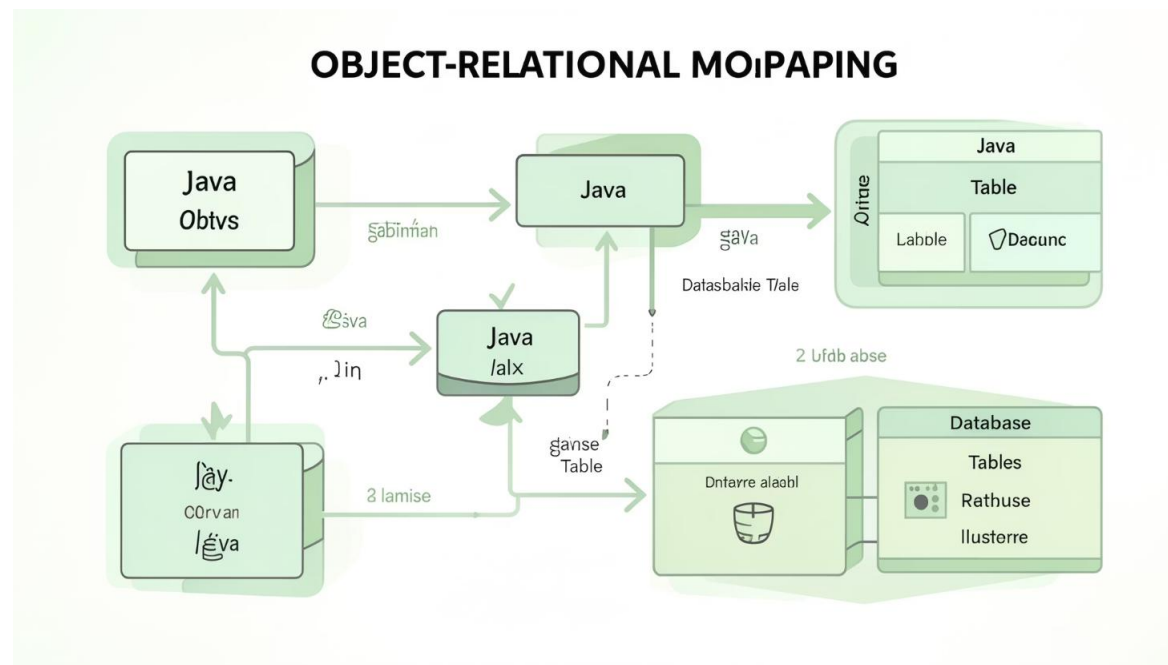
Đóng vai trò cầu nối giữa ứng dụng Java và database, che giấu chi tiết SQL



Tối ưu hiệu năng

Cung cấp caching, lazy loading và các kỹ thuật tối ưu truy vấn tự động

ORM – Object Relational Mapping



Khái niệm cơ bản

ORM là kỹ thuật ánh xạ dữ liệu giữa hệ thống quản trị cơ sở dữ liệu quan hệ và ngôn ngữ lập trình hướng đối tượng.

Thành phần chính

- **Entity:** Class Java đại diện cho một bảng
- **Fields:** Thuộc tính ánh xạ với cột
- **Relationships:** Quan hệ giữa các entity

Entity và Mapping

1

Annotation @Entity

Đánh dấu class là một entity có thể ánh xạ với bảng trong database

```
@Entity
public class User {
    // properties
}
```

2

@Table và @Id

Xác định tên bảng và trường khóa chính trong database

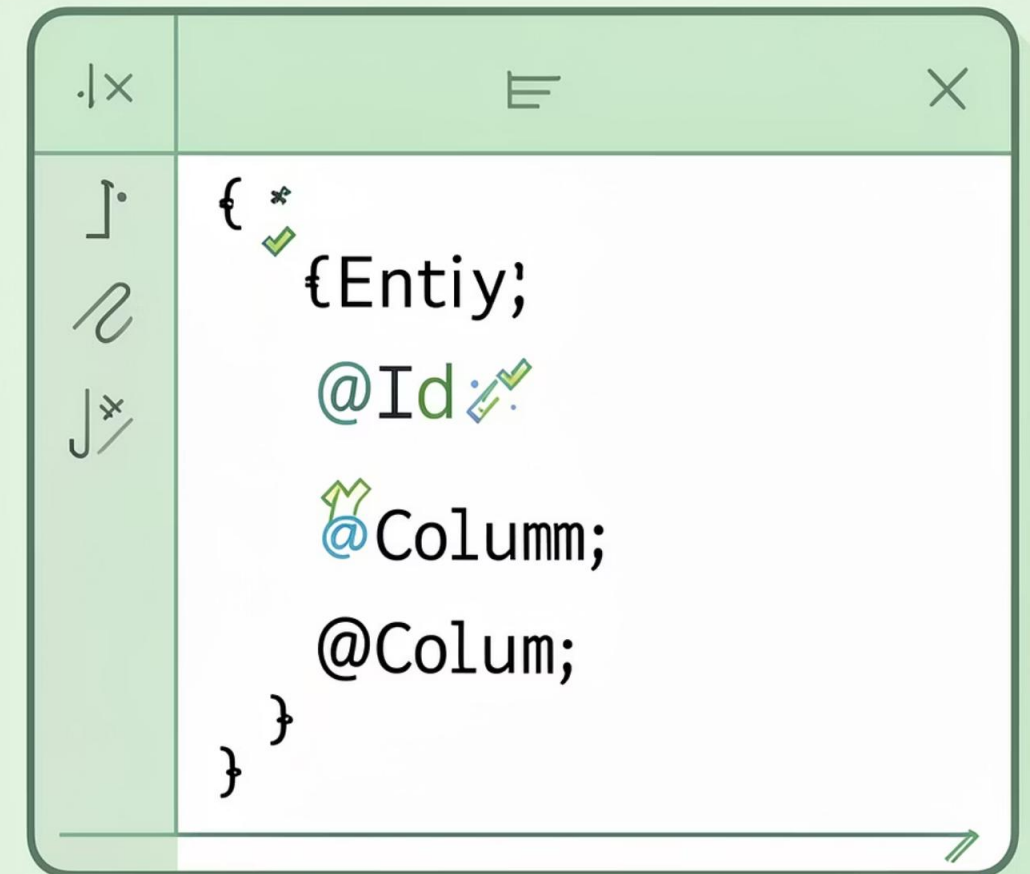
```
@Entity
@Table(name = "users")
public class User {
    @Id
    private Long id;
}
```

3

@Column

Ánh xạ field Java với column trong bảng, có thể chỉ định tên và thuộc tính

```
@Column(name = "username", nullable = false)
private String username;
```



Session và SessionFactory

SessionFactory

Tính chất:

- Thread-safe và immutable
- Tạo từ cấu hình Hibernate
- Duy nhất cho mỗi ứng dụng
- Tốn chi phí khởi tạo

Vai trò: Factory để tạo Session mới

- 📄 SessionFactory được tạo một lần và sử dụng xuyên suốt ứng dụng

Session

Tính chất:

- Không thread-safe
- Được tạo từ SessionFactory
- Phạm vi ngắn (per request)
- Rẻ để tạo và hủy

Vai trò: Giao diện chính cho persistence operations

- 📄 Session đại diện cho một phiên làm việc với database

Lấy Session từ SessionFactory

SessionFactory

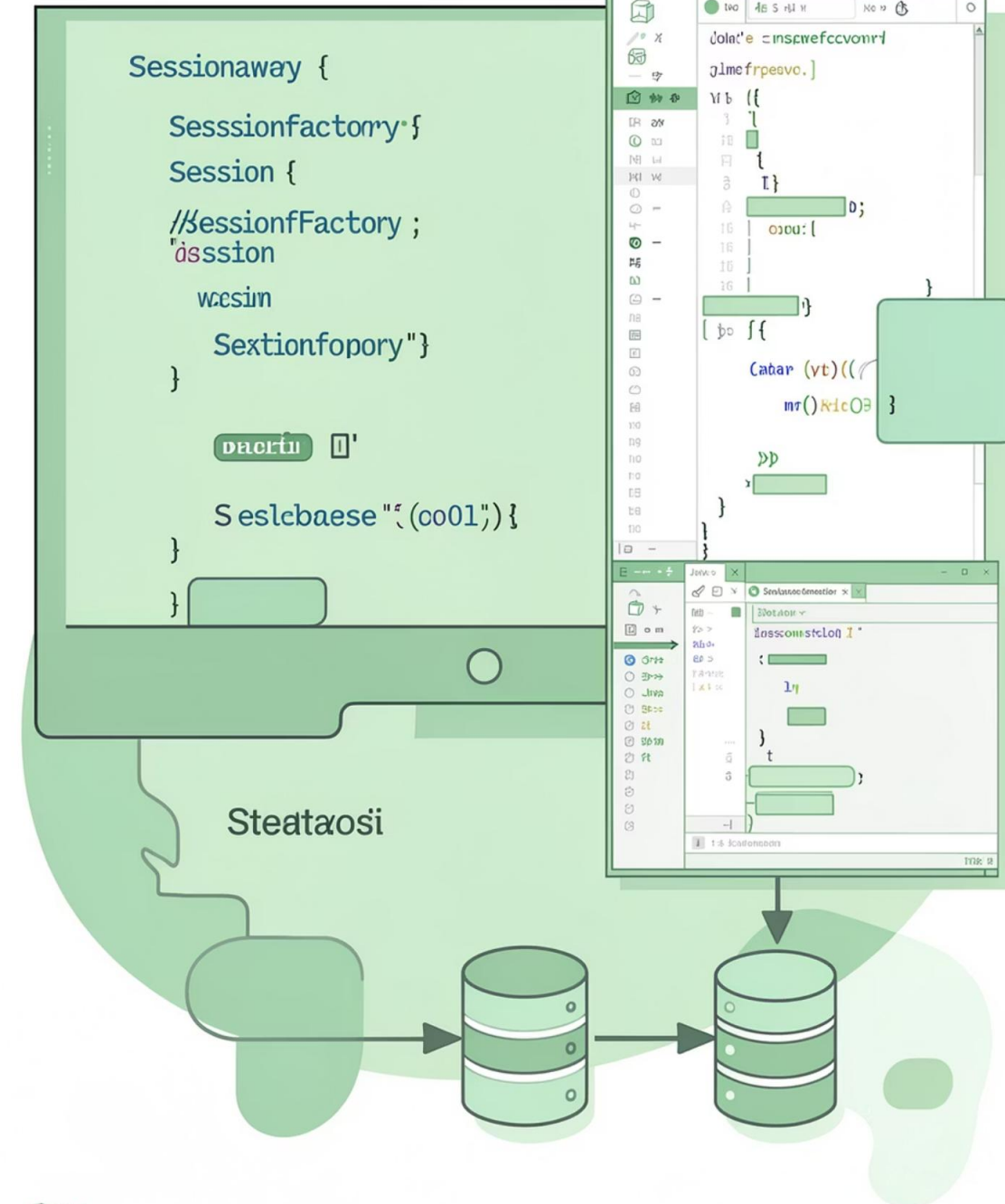
```
Configuration config =  
    new Configuration();  
config.configure(  
    "hibernate.cfg.xml"  
);  
  
SessionFactory factory =  
    config.buildSessionFactory()  
;
```

Session

```
Session session =  
    factory.openSession();  
  
try {  
    // Thực hiện các thao tác  
    // với database  
} finally {  
    session.close();  
}
```

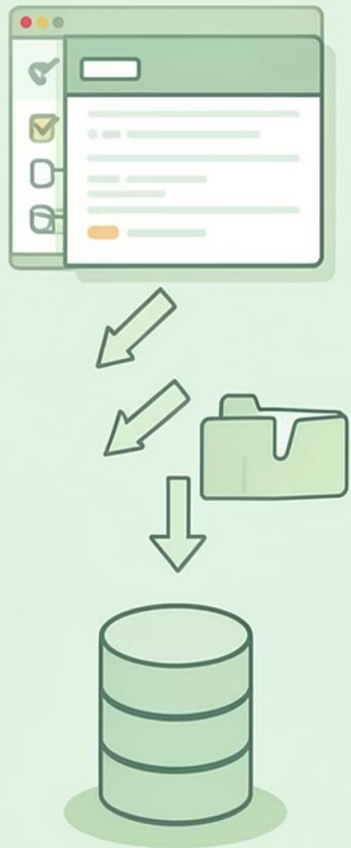
Java

SessionFactory and Session



Lazy Loading

Data only when needed.



Eager Loading

Regressing all related data at once



Lazy vs Eager Loading

Eager Loading

`@OneToMany(fetch = FetchType.EAGER)`

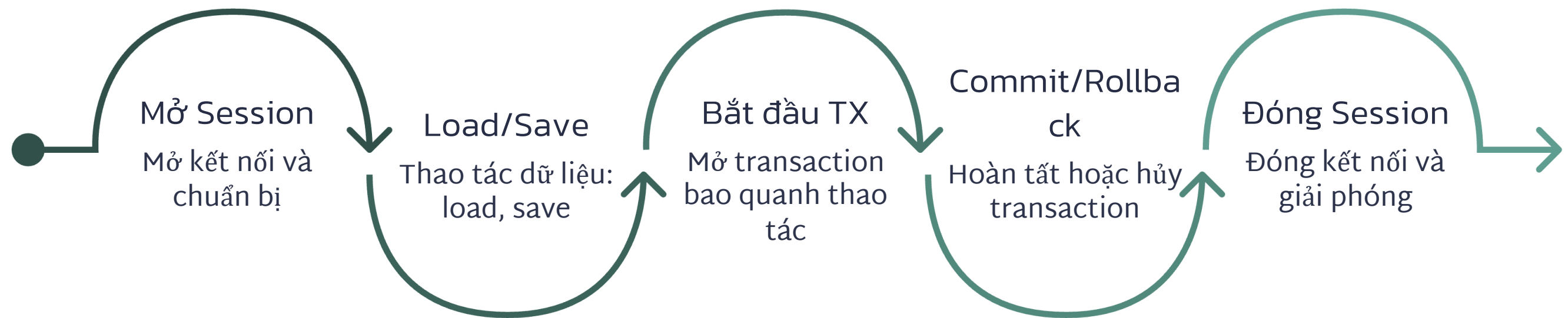
- Tải tất cả dữ liệu liên quan ngay lập tức
- Sử dụng JOIN trong SQL
- Phù hợp khi luôn cần dữ liệu
- Tăng memory usage ban đầu

Lazy Loading

`@OneToMany(fetch = FetchType.LAZY)`

- Tải dữ liệu khi thực sự truy cập
- Truy vấn bổ sung khi cần
- Phù hợp với dữ liệu lớn
- Giảm memory usage ban đầu

Session Lifecycle



Hibernate vs JDBC vs JPA

Tiêu chí	JDBC	Hibernate	JPA
Loại	API chuẩn	Framework ORM	Specification
SQL	Viết thủ công	Tự động sinh	Native/JPQL
Object mapping	Không có	Tự động	Có annotation
Caching	Không	Có (L1, L2)	Phụ thuộc implement
Transaction	Manual	Hỗ trợ mạnh	Annotation
Vendor lock-in	Có	Giảm	Không
Độ phức tạp	Thấp	Trung bình	Trung bình

 **Ghi chú:** JPA là specification, Hibernate là một implementation của JPA specification. Có thể dùng cả hai cùng nhau để tận dụng annotation của JPA và tính năng của Hibernate.

Cấu Hình Hibernate

01

hibernate.cfg.xml

Tập tin cấu hình chính chứa thông tin kết nối cơ sở dữ liệu, driver, username, password và các cài đặt khác

02

SessionFactory

Được tạo từ cấu hình, dùng để tạo các phiên làm việc với cơ sở dữ liệu

03

Session

Phiên làm việc cụ thể, cung cấp các phương thức CRUD để tương tác với cơ sở dữ liệu

Annotation Cơ Bản



@Entity

Đánh dấu lớp Java là một thực thể được ánh xạ với bảng cơ sở dữ liệu



@Id

Chỉ định trường là khóa chính của bảng



@Column

Ánh xạ trường với cột cụ thể trong bảng

```
1  Java Class(Jass);
2  <|/'ya |>· |> (<|> |>· > >>>
3  { ava chal = Y) }
9  hon!"lt0 }
3  manice class' = (lat'(y))
5  fpyeding túsde) }
6  /incire = faofclo((y|)}"
7  { last carplapl(" gtaKls)bx}
4  Pave carff:xt (>{
11  Mojdidu "tlve (im" K);
9  Pava se poza(nts{   };
12  Davt er tnapr(lmsse);
5  Padt e> (napr(lmest);
4  Randioo serip), > /);   )
4  hfodlovs'}
12  Pat(ronntef) = /o(ðap);
Mt
R
D
41  }
20 }
```

CRUD: Create & Read

Create – Tạo mới

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

User user = new User();
user.setName("Nguyen Van A");
user.setEmail("a@gmail.com");

session.save(user);
tx.commit();
session.close();
```

Read – Đọc dữ liệu

```
Session session = factory.openSession();

User user = session.get(User.class, 1L);
System.out.println(user.getName());

session.close();
```

CRUD: Update & Delete

Update – Cập nhật

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

User user = session.get(User.class, 1L);
user.setName("Updated Name");

session.update(user);
tx.commit();
session.close();
```

Delete – Xóa

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();

User user = session.get(User.class, 1L);
session.delete(user);

tx.commit();
session.close();
```

Quản Lý Giao Dịch (Transaction)

1

`beginTransaction()`

Bắt đầu giao dịch mới và mở một khối lệnh

2

`commit()`

Ghi các thay đổi vào cơ sở dữ liệu vĩnh viễn

3

`rollback()`

Hoàn tác tất cả thay đổi nếu có lỗi xảy ra

 **ACID:** Atomicity (nguyên tử), Consistency (nhất quán), Isolation (cô lập), Durability (bền vững)

Giao Dịch Với Try-Catch

```
Session session = factory.openSession();
Transaction tx = null;

try {
    tx = session.beginTransaction();

    User user = new User();
    user.setName("Test User");
    session.save(user);

    int result = 10 / 0; // Exception

    tx.commit();
} catch (Exception e) {
    if (tx != null) {
        tx.rollback();
    }
    e.printStackTrace();
} finally {
    session.close();
}
```

Giải thích

- Nếu có lỗi xảy ra, rollback sẽ được gọi tự động
- Các thay đổi không được ghi vào cơ sở dữ liệu
- Đảm bảo tính toàn vẹn dữ liệu

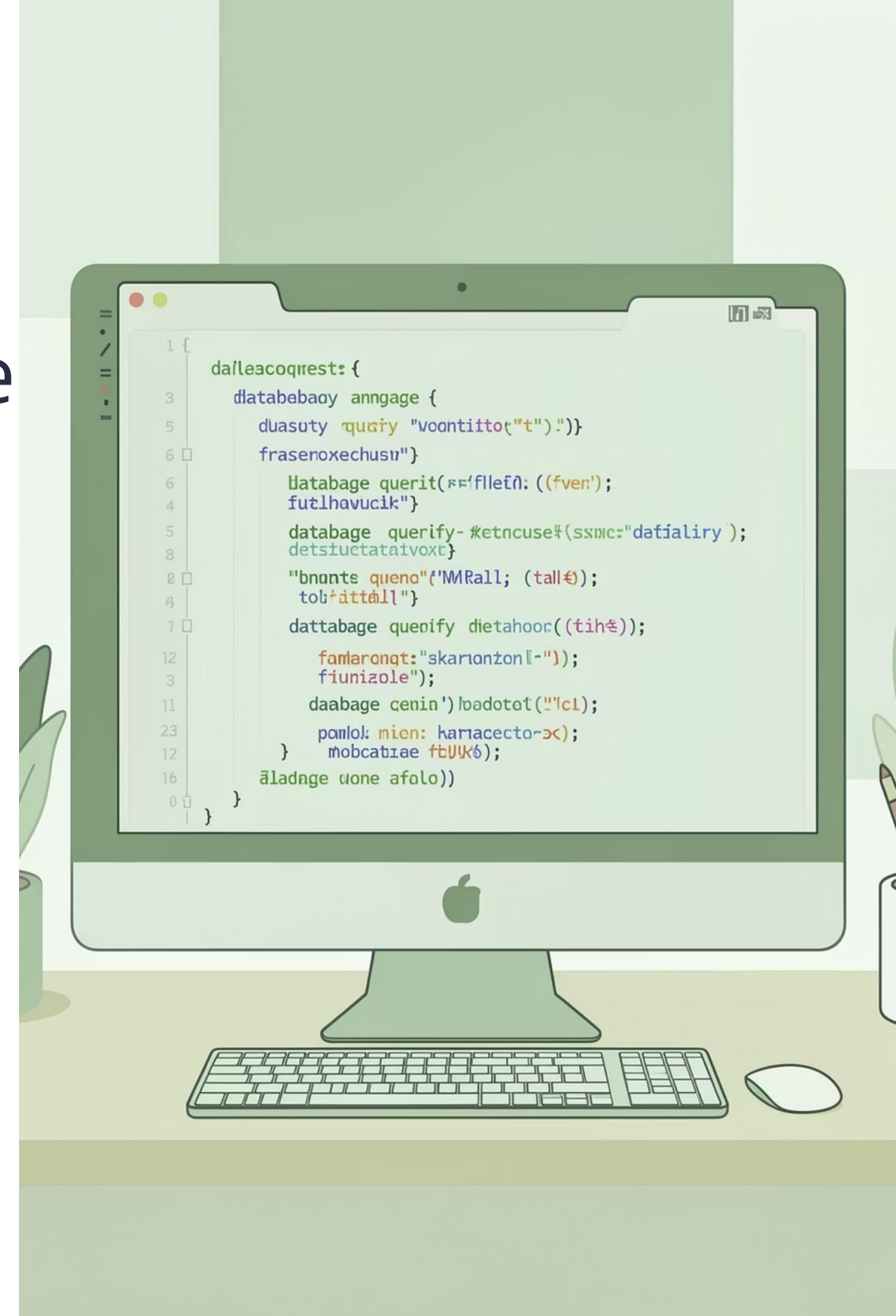
HQL – Hibernate Query Language

HQL là gì?

Ngôn ngữ truy vấn giống SQL nhưng làm việc với các thực thể Java thay vì bảng cơ sở dữ liệu

SQL vs HQL

- SQL: truy vấn bảng và cột
- HQL: truy vấn lớp và trường
- HQL hỗ trợ tính kế thừa và liên kết



Ví Dụ Truy Vấn HQL

Truy vấn tất cả người dùng

```
String hql = "FROM  
User";  
Query query =  
session.createQuery(hql)  
;  
List users =  
query.list();
```

Truy vấn có điều kiện

```
String hql = "FROM User  
WHERE name = :name";  
Query query =  
session.createQuery(hql)  
;  
query.setParameter("name",  
"Nguyen Van A");  
List users =  
query.list();
```

Truy vấn với JOIN

```
String hql = "FROM User  
u JOIN u.posts p WHERE  
p.title LIKE :title";  
Query query =  
session.createQuery(hql)  
;  
query.setParameter("title",  
"%Java%");  
List results =  
query.list();
```

Case Study: Hệ thống quản lý sinh viên

01

Tạo Entity Student

Định nghĩa lớp Student với các thuộc tính id, name, age

02

Thiết lập Session Factory

Kết nối với cơ sở dữ liệu thông qua cấu hình

03

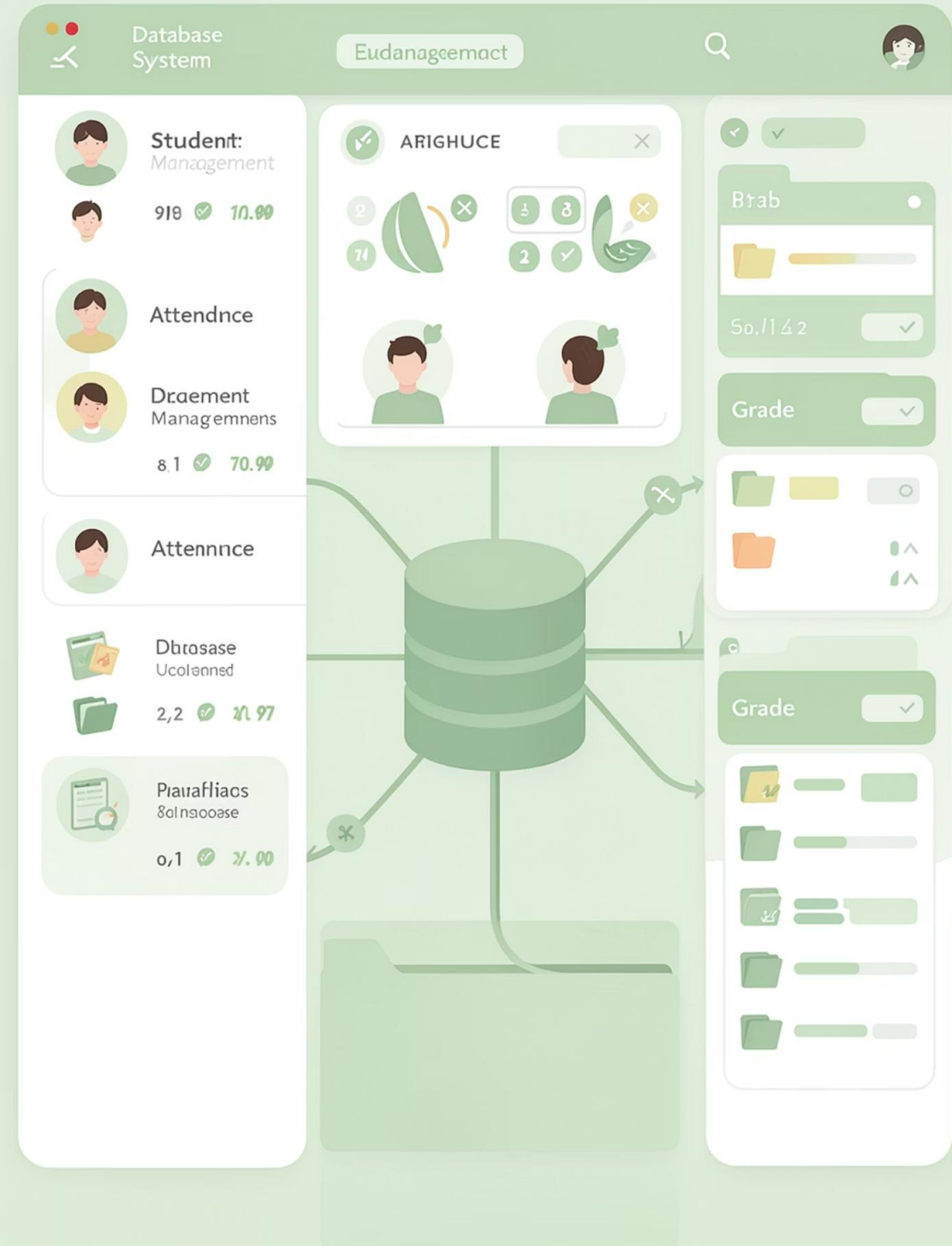
Thực hiện CRUD

Thêm, sửa, xóa và tìm kiếm sinh viên

04

Truy vấn với HQL

Sử dụng Hibernate Query Language để tìm kiếm nâng cao



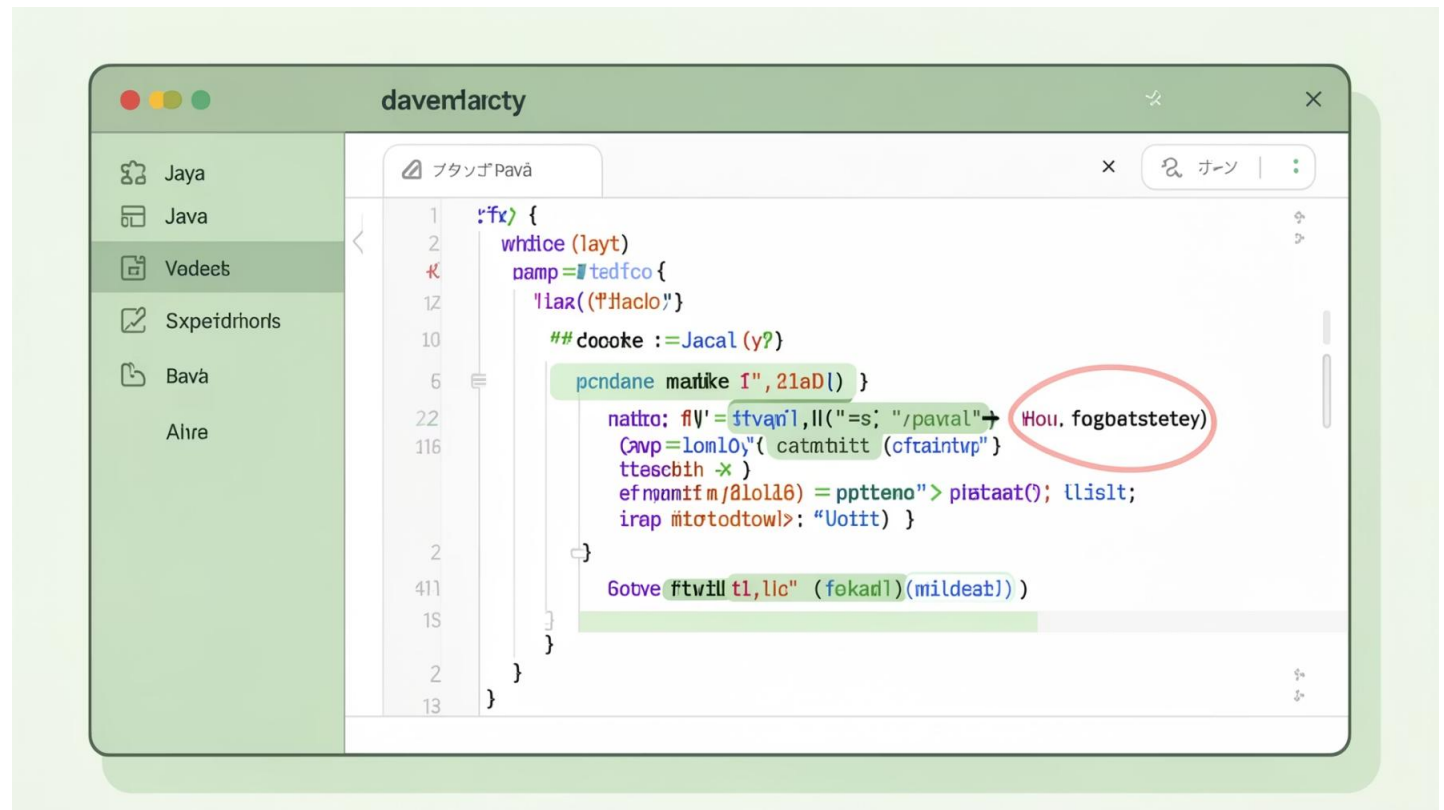
Entity Student

```
@Entity
@Table(name = "students")
public class Student {
    @Id
    @GeneratedValue
    private Long id;

    @Column(name = "name")
    private String name;

    @Column(name = "age")
    private int age;

    // Constructors, getters, setters
}
```



- ❏ **@Entity**: Đánh dấu lớp là entity
- @Table**: Chỉ định tên bảng
- @Id**: Khóa chính
- @GeneratedValue**: Tự động sinh ID

CRUD Operations



Create

```
session.save(student)
```



Read

```
session.get(Student.class, id)
```



Update

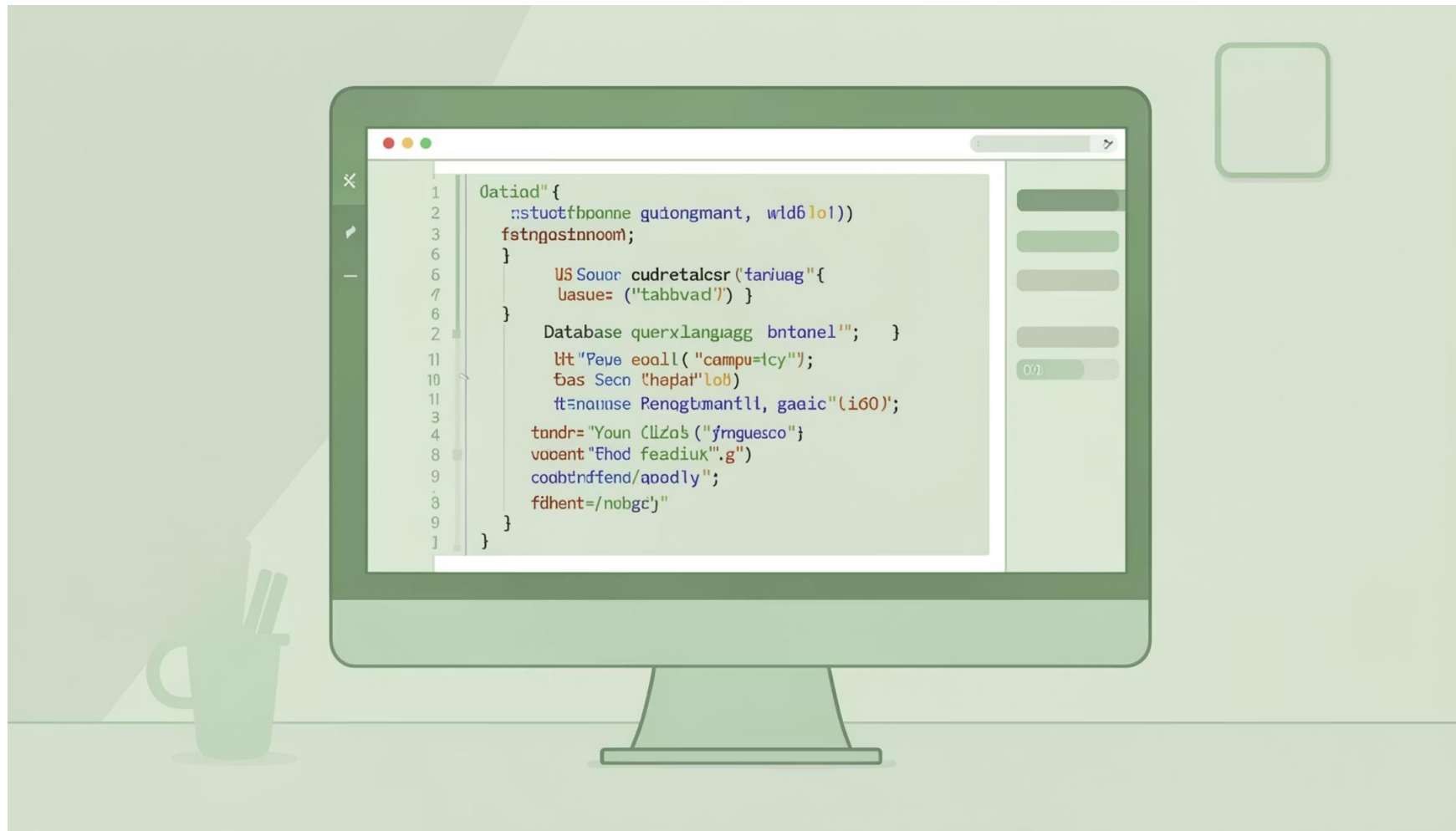
```
session.update(student)
```



Delete

```
session.delete(student)
```

HQL – Hibernate Query Language



Truy vấn cơ bản

```
String hql = "FROM Student";  
Query query =  
session.createQuery(hql);  
List students = query.list();
```

Truy vấn có điều kiện

```
String hql = "FROM Student  
WHERE age > :minAge";  
Query query =  
session.createQuery(hql);  
query.setParameter("minAge",  
18);  
List students = query.list();
```

Best Practices

Quản lý Session

- Sử dụng try-with-resources
- Đóng session sau khi sử dụng
- Tránh giữ session mở quá lâu

Tránh N+1 Query

- Sử dụng JOIN FETCH trong HQL
- Fetch type EAGER cẩn thận
- Batch fetching để tối ưu

Lazy Loading đúng cách

- Fetch type LAZY cho quan hệ
- Initialize collections khi cần
- Hiểu rõ khi nào dữ liệu được tải

Câu hỏi thảo luận

1 So sánh Hibernate và JDBC

Hibernate có những ưu điểm gì so với JDBC thuần? Khi nào nên dùng JDBC thay vì Hibernate?

2 Lazy vs Eager Loading

Khi nào nên sử dụng lazy loading? Những rủi ro tiềm ẩn của lazy loading là gì?

3 Cache trong Hibernate

Giải thích cơ chế cache first-level và second-level trong Hibernate. Cách nào để tối ưu hiệu suất?

4 Giao dịch (Transaction)

Tại sao cần bao bọc các operation trong transaction? Hậu quả nếu không sử dụng transaction?

5 Hiệu suất ứng dụng

Ngoài tránh N+1 query, còn những cách nào để tối ưu hiệu suất của ứng dụng Hibernate?



THANK YOU

Cảm ơn!

Tài liệu tham khảo

Hibernate Documentation và
tutorials

Thực hành

Xây dựng project với các
entity và quan hệ

Hỏi đáp

Đặt câu hỏi và thảo luận thêm